

Disponibilidad (Availability)

Disponibilidad

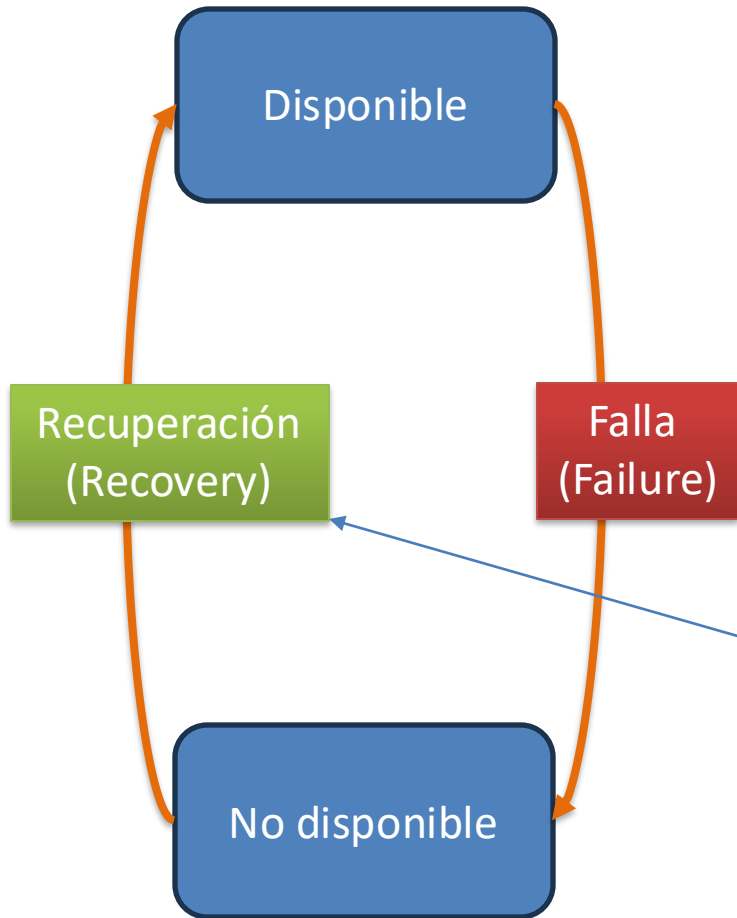
- ¿Qué es Disponibilidad?
- Medición
- Downtime
- Tácticas
- Patrones

Disponibilidad

- El sistema está “siempre listo” para llevar a cabo sus tareas cuando sea que se requiera

Disponibilidad (Availability)

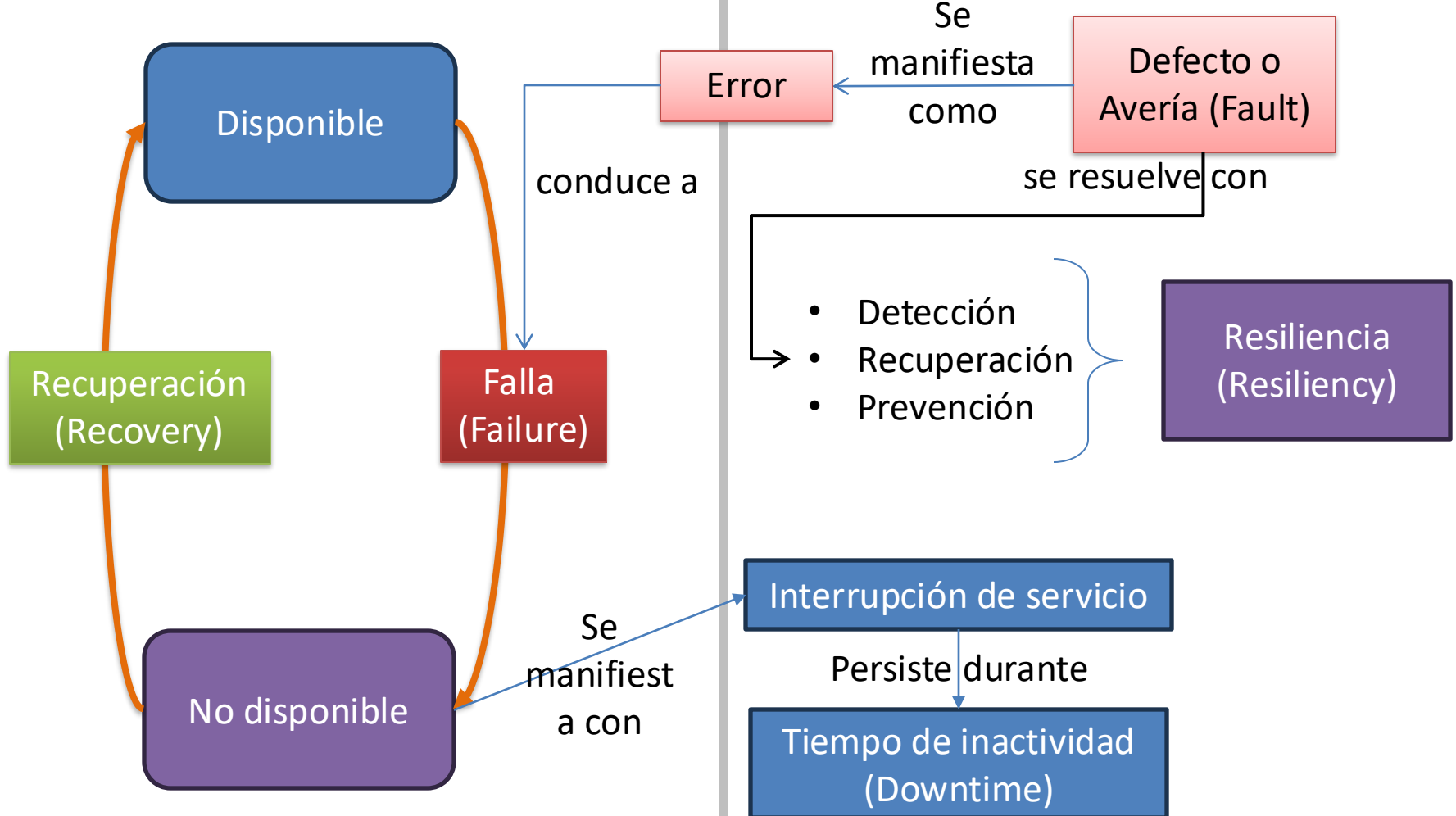
Sistema, Contenedor, Componente



- Minimizar probabilidades de Falla
- Recuperación en el menor tiempo posible

Disponibilidad (Availability)

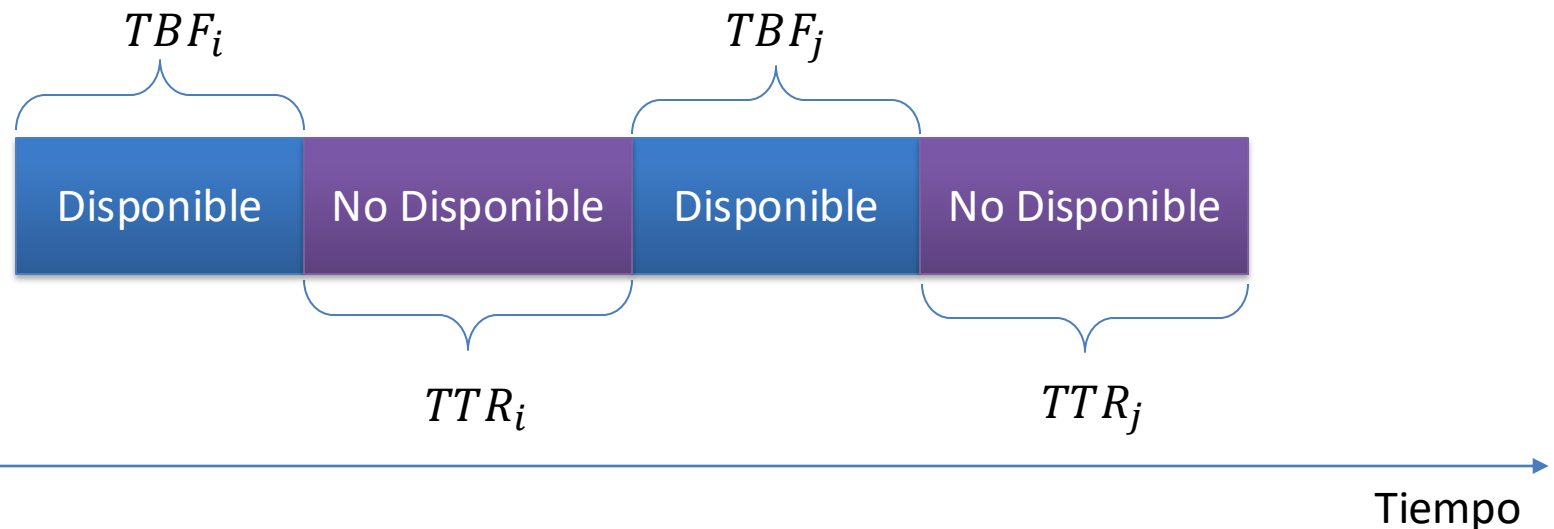
Sistema, Contenedor, Componente



Fórmula de Disponibilidad

$$\frac{MTBF}{MTBF + MTTR}$$

- *MTBF*: Mean time between failures
- *MTTR*: Mean time to repair



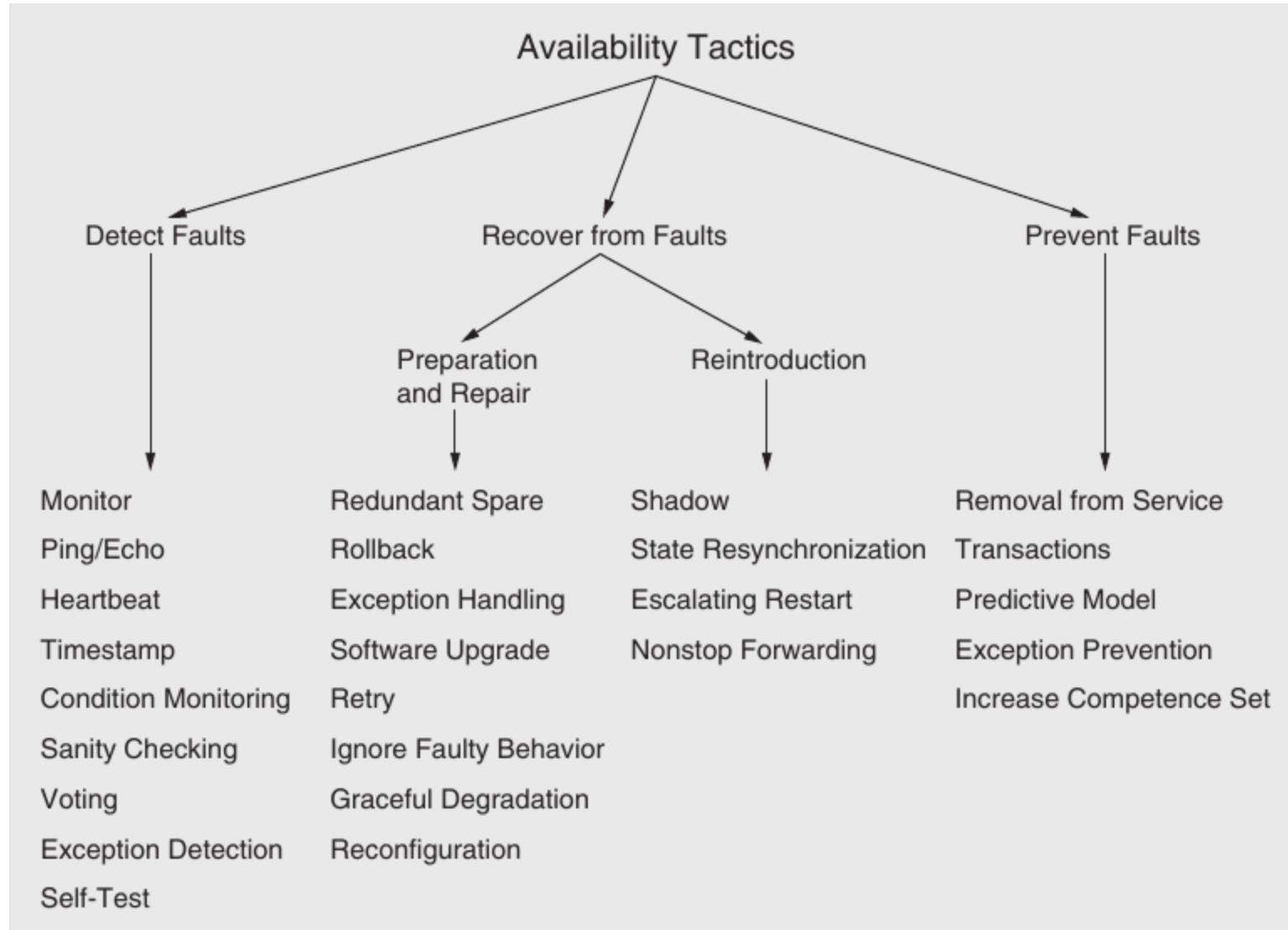
Service Level Agreement (SLA)

- Acuerdo de Nivel de Servicio
 - Incluye compromiso de Disponibilidad
- ¿Qué quiere decir 99.99% de disponibilidad?
0.01% de probabilidad que el sistema no esté disponible cuándo se necesita

Availability	Downtime/90 Days	Downtime/Year
99.0%	21h 36m	3 days, 15.6h
99.9%	2h 10m	8h 0m 46s
99.99%	12m 58s	52m 34s
99.999%	1m 18s	5m 15s
99.9999%	8s	32s

- Mantenimientos programados
 - ¿Deben o no contar para la fórmula de disponibilidad?

Tácticas para Disponibilidad



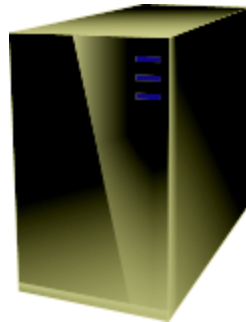
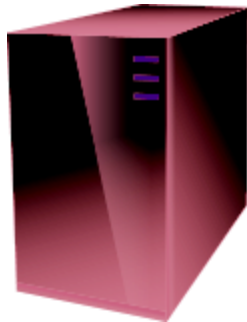
Detección de Defectos

- Monitoreo
- Ping/echo
- Heartbeat (Latido o Pulso)
- Timestamp (Marca de tiempo)
- Monitoreo de condiciones:
- Sanity checking (Verificación de sanidad)
- Voting (Votación)
- Detección de excepciones
- Self-test

Monitorio

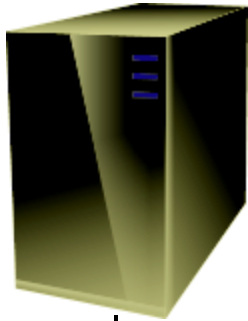


- CPU
- Memoria
- I/O
- Etc.



Ping / Echo

Monitor



Sistema, contenedor, componente



Mensaje



Respuesta



Mensaje



Respuesta



...

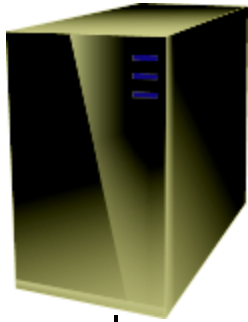
Se recibió respuesta?



Heartbeat (Latido o pulso)

Monitor

Sistema, contenedor, componente



Pulso (Heartbeat)



Pulso (Heartbeat)

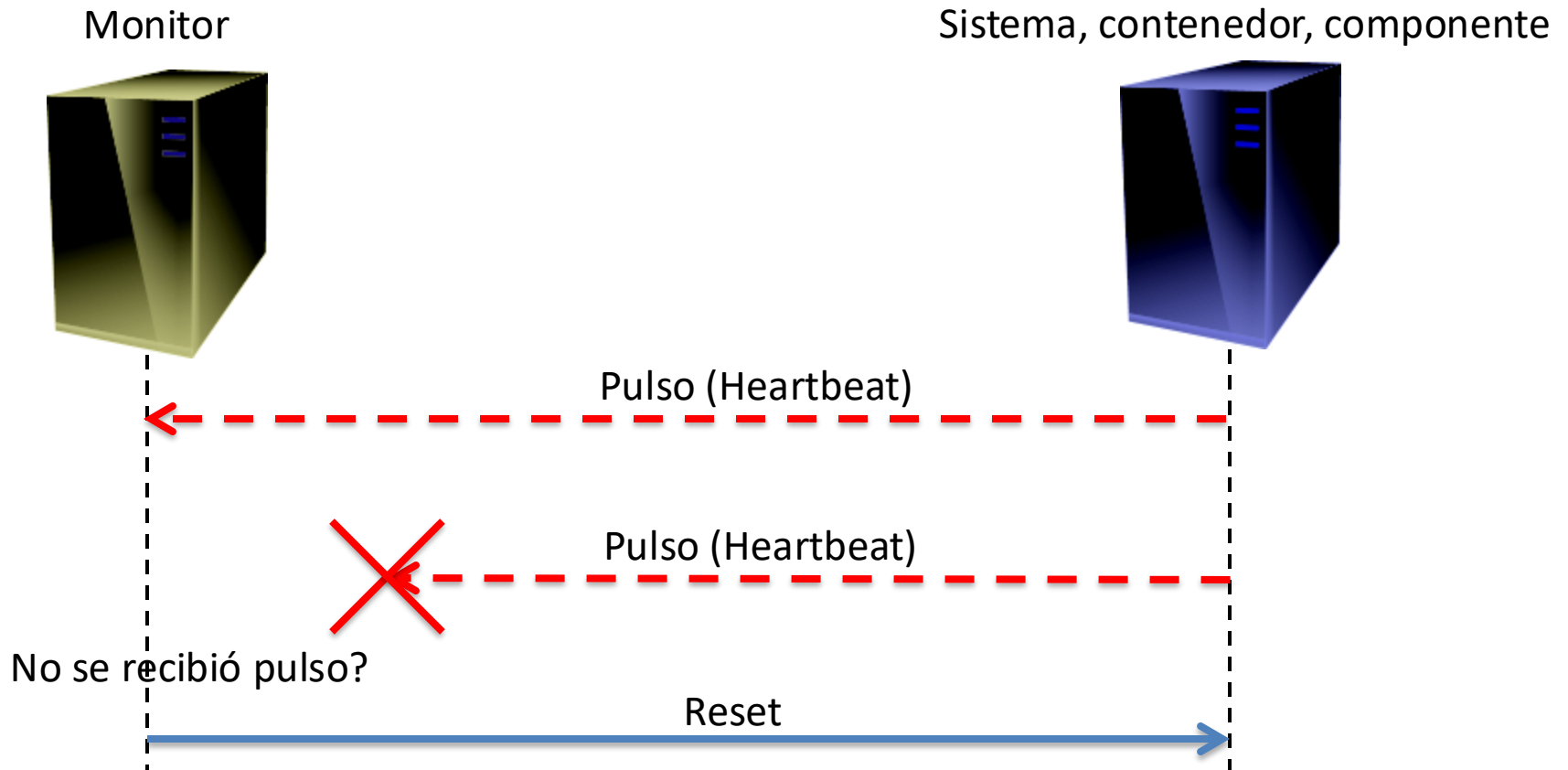


...

Se recibió pulso?



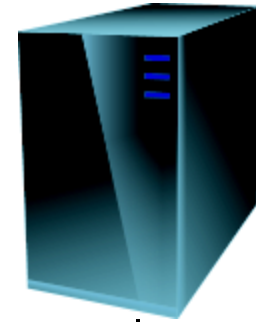
Heartbeat - Watchdog



Timestamp (Marca de tiempo)

Sistema, contenedor, componente

Receptor



Mensaje + T1

Mensaje + T2

...

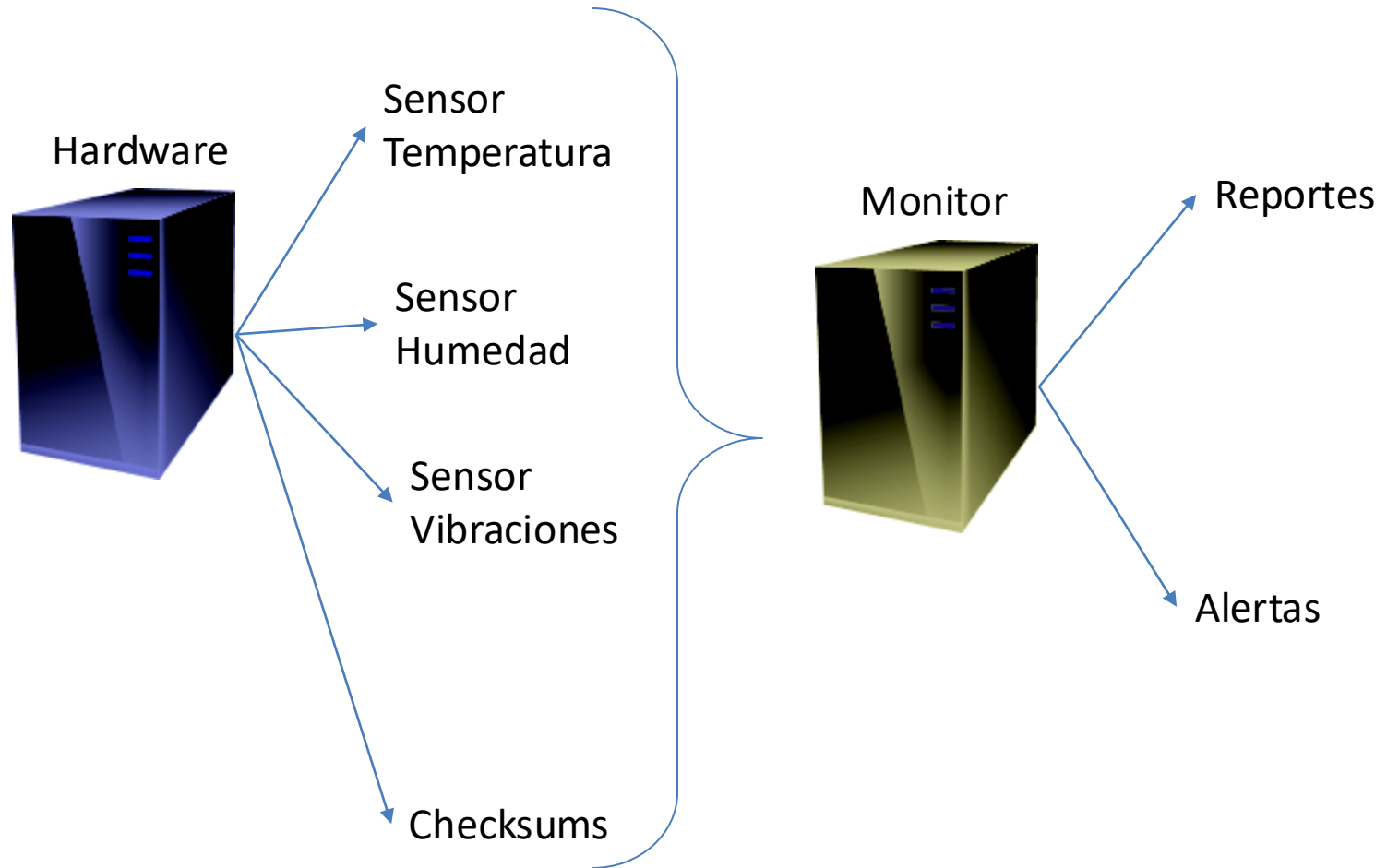
Mensaje + Tn

Timestamp
O consecutivo

Se recibieron los
mensajes en el orden
correcto?



Monitoreo de condiciones

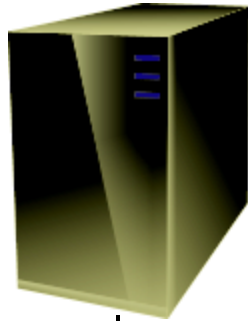


<https://www.thegamer.com/how-ionizing-particle-outer-space-helped-super-mario-64-speedrunner-save-time/>

<https://www.youtube.com/watch?v=bhBf5crp0i8>

Sanity checking (Verificación de sanidad)

Monitor

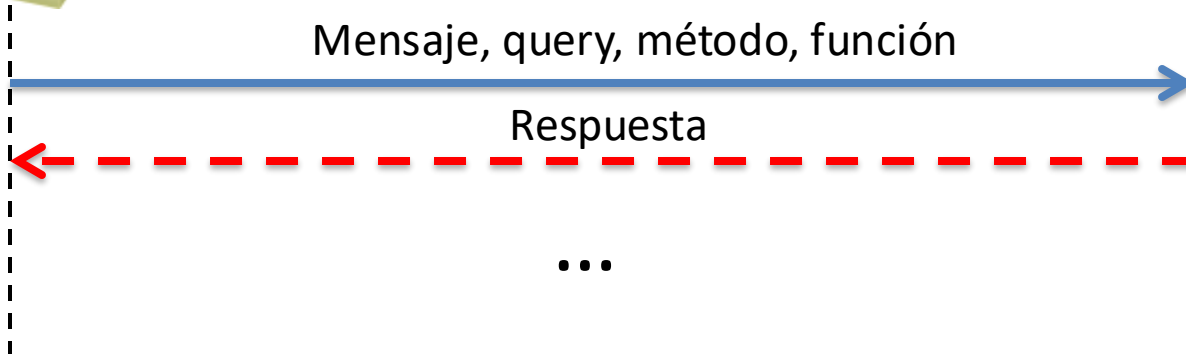


Sistema, contenedor, componente



Mensaje, query, método, función

Respuesta

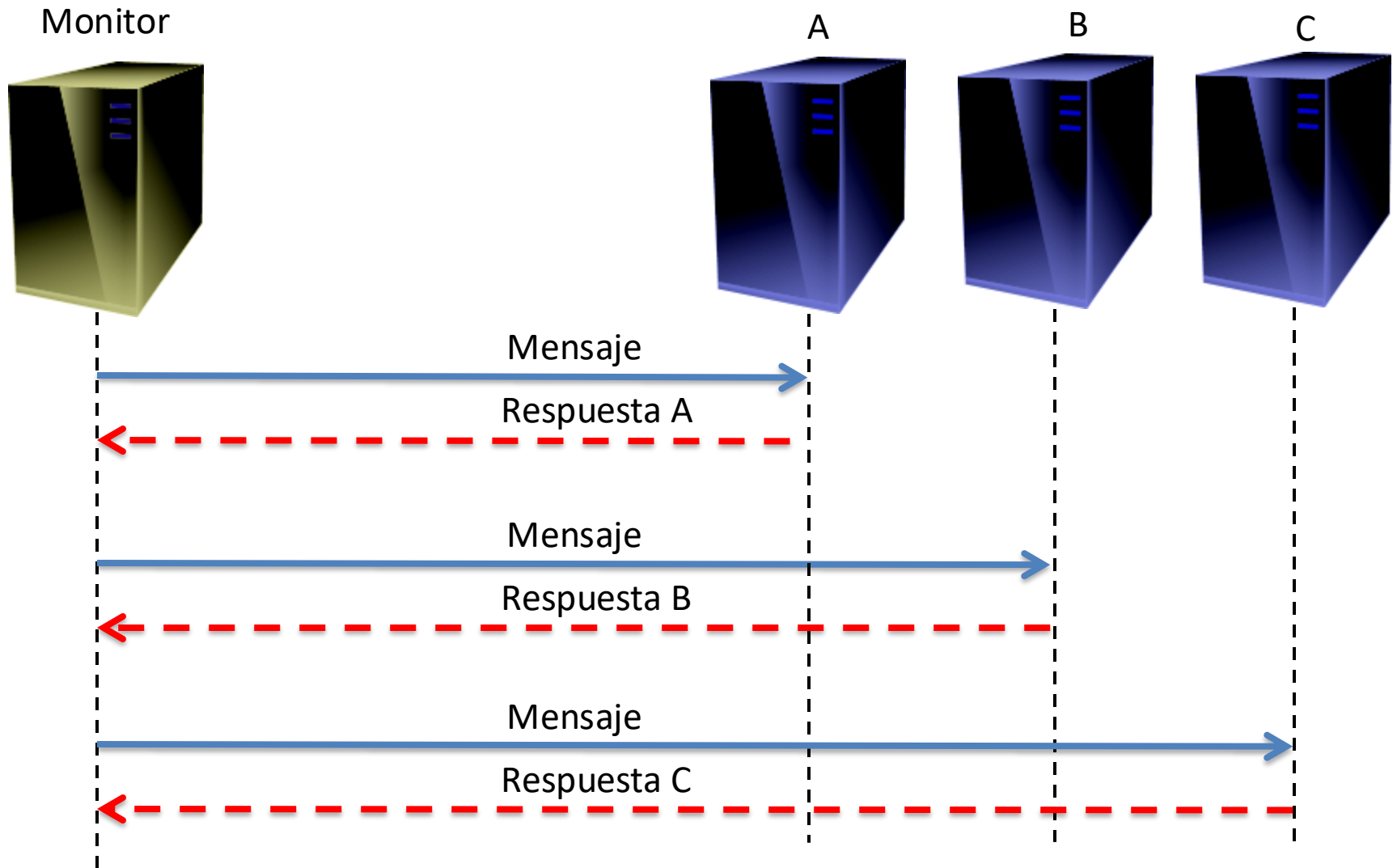


La respuesta es correcta
de acuerdo al diseño del
sistema?



- Arroja resultados numéricos correctos?
- Consulta correctamente algún dato de la BD?
- Retorna sin errores o excepciones?

Voting (Votación)



Coinciden las respuestas de las réplicas?



Detección de excepciones

- Capturar excepciones de sistema (try-catch)
- Tipado fuerte sobre parámetros
 - Especialmente si se extraen de strings
 - Ejemplo: parámetros de métodos REST en Spring Boot
- Timeout: lanzar y manejar excepciones cuando un componente no responde a tiempo

```
@GetMapping("/{id}")  
@Transactional  
public Cuenta cuenta(@PathVariable Long id) {  
    return cuentaService.cuenta(id);  
}
```

Self-test

- Un componente ejecuta periódicamente un programa para probarse a sí mismo
 - Complementario a Sanity check y Heartbeat
 - Pruebas de hardware
 - Checksums
- Ejemplos
 - BIOS Self-test
 - Routers y Switches: prueban que la memoria esté ok y las tarjetas de red, conectadas y funcionando
 - Colas de mensaje: mensaje de prueba “canario”, para ver si la cola transmite datos
 - Verificaciones de integridad (checksums) en gestores bases de datos

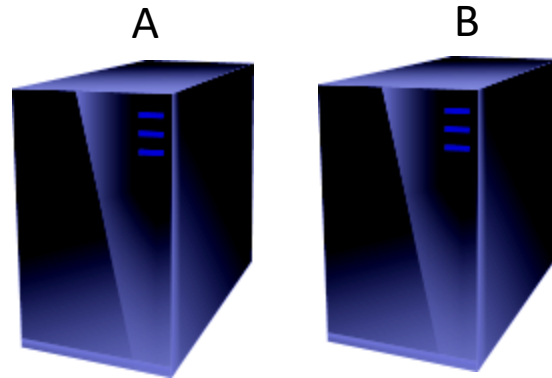
Recuperación de Defectos

Reparación y Reintroducción

Reparación

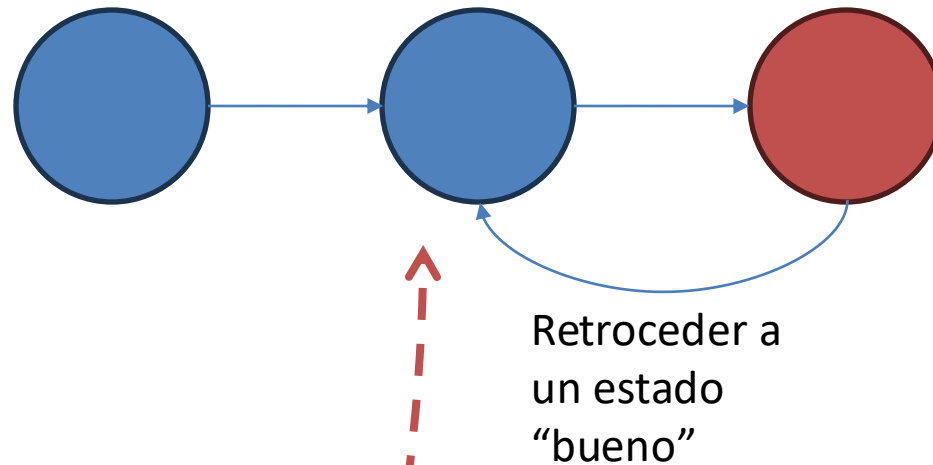
- Redundant spare (Repuesto redundante) / Réplicas
- Rollback
- Manejo de excepciones
- Actualización de software
- Retry (Reintentar)
- Ignorar comportamiento defectuoso
- Graceful degradation (Degradación elegante)
- Reconfiguración

Redundant spare (repuesto redundante)



- **Redundant spare:** Si el componente A falla → el componente B toma su lugar

Rollback



- Rollbacks de BD
- Recuperación de sistemas de archivo
 - Journaling Filesystems
- Checkpoints

Manejo de excepciones

```
Exception in thread "main" java.lang.NullPointerException
    at org.flowable.engine.impl.bpmn.behavior.DmnActivityBehavior.execute(DmnActivityBehavior.java:101)
    at org.flowable.engine.impl.agenda.ContinueProcessOperation.executeActivityBehavior(ContinueProcessOperation.java:264)
    at org.flowable.engine.impl.agenda.ContinueProcessOperation.executeSynchronous(ContinueProcessOperation.java:158)
    at org.flowable.engine.impl.agenda.ContinueProcessOperation.continueThroughFlowNode(ContinueProcessOperation.java:113)
    at org.flowable.engine.impl.agenda.ContinueProcessOperation.continueThroughSequenceFlow(ContinueProcessOperation.java:312)
    at org.flowable.engine.impl.agenda.ContinueProcessOperation.run(ContinueProcessOperation.java:79)
    at org.flowable.engine.impl.interceptor.CommandInvoker.executeOperation(CommandInvoker.java:88)
    at org.flowable.engine.impl.interceptor.CommandInvoker.executeOperations(CommandInvoker.java:72)
    at org.flowable.engine.impl.interceptor.CommandInvoker.execute(CommandInvoker.java:56)
    at org.flowable.engine.impl.interceptor.BpmnOverrideContextInterceptor.execute(BpmnOverrideContextInterceptor.java:25)
    at org.flowable.common.engine.impl.interceptor.TransactionContextInterceptor.execute(TransactionContextInterceptor.java:53)
    at org.flowable.common.engine.impl.interceptor.CommandContextInterceptor.execute(CommandContextInterceptor.java:71)
    at org.flowable.common.engine.impl.interceptor.LogInterceptor.execute(LogInterceptor.java:35)
    at org.flowable.common.engine.impl.cfg.CommandExecutorImpl.execute(CommandExecutorImpl.java:56)
    at org.flowable.common.engine.impl.cfg.CommandExecutorImpl.execute(CommandExecutorImpl.java:51)
    at org.flowable.engine.impl.TaskServiceImpl.complete(TaskServiceImpl.java:213)
    at ma.cr.courrier.CourrierArriveeTestDMNTest.FindAndTransitTo(CourrierArriveeTestDMNTest.java:151)
    at ma.cr.courrier.CourrierArriveeTestDMNTest.main(CourrierArriveeTestDMNTest.java:129)
```

- ¿Cómo reacciona el sistema cuando ocurre una excepción?
 - Crash
 - Mandar mensaje de error

Actualización de software

- Actualizar software sin afectar su ejecución
- Function / class patch
 - Hotfixes
- In-service software upgrade (ISSU)
 - Combinar con redundant spare

```
public class CourseSection {  
    public class CourseSection {  
        public class CourseSection {  
            public class CourseSection {  
                public CourseDescription getDescription() {  
                    <retrieve course description>  
                }  
                public Integer getTerm() {  
                    <retrieve term>  
                } ... }  
            }  
        }  
    }  
}
```



Retry (Reintentar)

- Ejecutar nuevamente la acción que arrojó error
- Máximo número de reintentos
- ¿Bajo qué condiciones esto puede funcionar?
 - Protocolos de red, tienen mecanismos específicos para manejar reenvío de paquetes
 - Operaciones idempotentes

Ignorar comportamiento defectuoso

- No hacer nada en respuesta a mensajes de error
 - Los mensajes no representan errores “reales”
 - El sistema no se ve afectado por el error
 - Es más una advertencia que un error
- ¿Ejemplos?



Graceful degradation

(Degradación elegante)

- Mantener el funcionamiento de partes críticas
- Dar de baja o reducir desempeño de funciones menos críticas
- Ejemplos
 - Priorizar catálogo genérico de productos (en cache o estático), por sobre catálogo personalizado
 - Bajar calidad de video/audio en streaming, cuando hay mucha demanda o la red está lenta
 - Mensaje: “servicio temporalmente no disponible”, cuando aquella parte del sistema se cae

Reconfiguración

- Reasignar responsabilidades cuando parte del sistema falla
 - Desplegar contenedores en menos nodos
 - Graceful degradation
- Reintroducción de componentes después de ser restablecidos

Reintroducción

Shadow

- El componente se deja corriendo “oculto” para ver que funcione bien antes de reintroducirlo

Resincronización de estado

- Al momento de reincorporar un componente
 - Sincronizar estado con los componentes activos
 - Asegurar su estado esté al día respecto de los demás
- Ejemplo:
<https://www.postgresql.org/docs/current/high-availability.html>

Reconfiguración – Reintroducción de componentes restablecidos

Reinicio escalado

- Comenzar con las unidades más pequeñas
- Continuar progresivamente con unidades más grandes hasta que el problema se resuelva

Ejemplo

1. Reiniciar hilos del proceso
2. Reiniciar el proceso
3. Reiniciar el contendor
4. Reiniciar VM
5. Reiniciar máquina física

Reconfiguración – Reintroducción de componentes restablecidos

Nonstop Forwarding

- OSI Nivel 3
 - Control Plane
 - Detectar topología
 - Tablas de enrutamiento/forwarding
 - Calcular rutas óptimas
 - Manejo de tráfico
 - Data Plane
 - Lee las tablas del control plane para decidir donde enviar paquetes
 - Envía los paquetes
- Cuando el Control Plane está caído
- El Data Plane continua enviando paquetes a las rutas definidas previamente a que se cayera el Control Plane

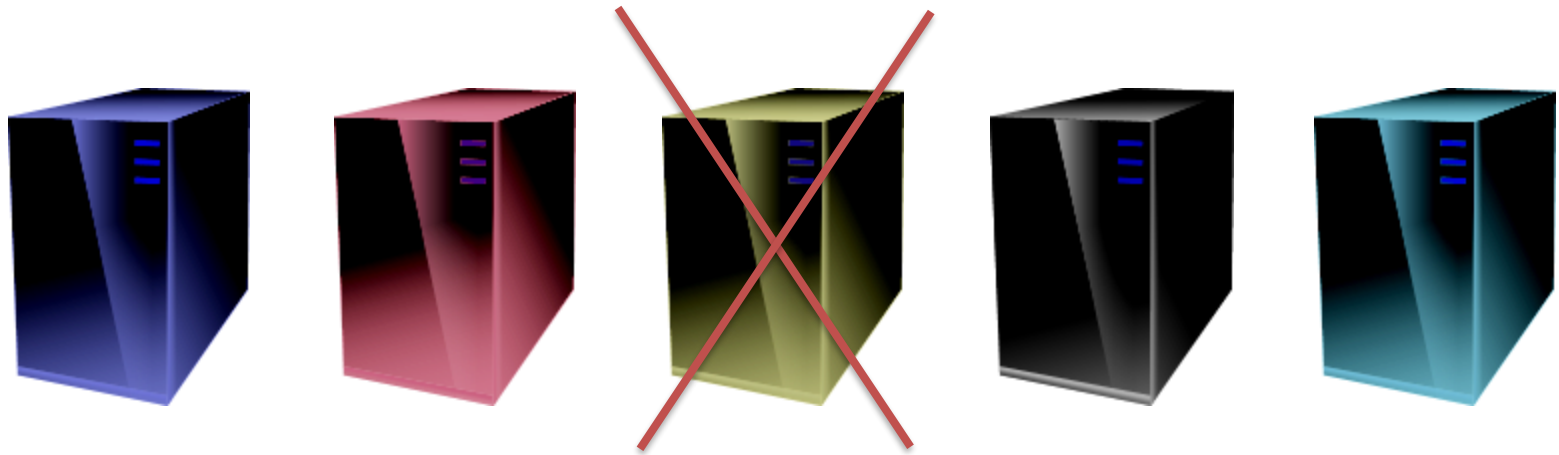
Prevención de Defectos

Prevención de Defectos

- Dar de baja
- Transacciones
- Modelo predictivo
- Prevención de excepciones
- Incrementar estados competentes

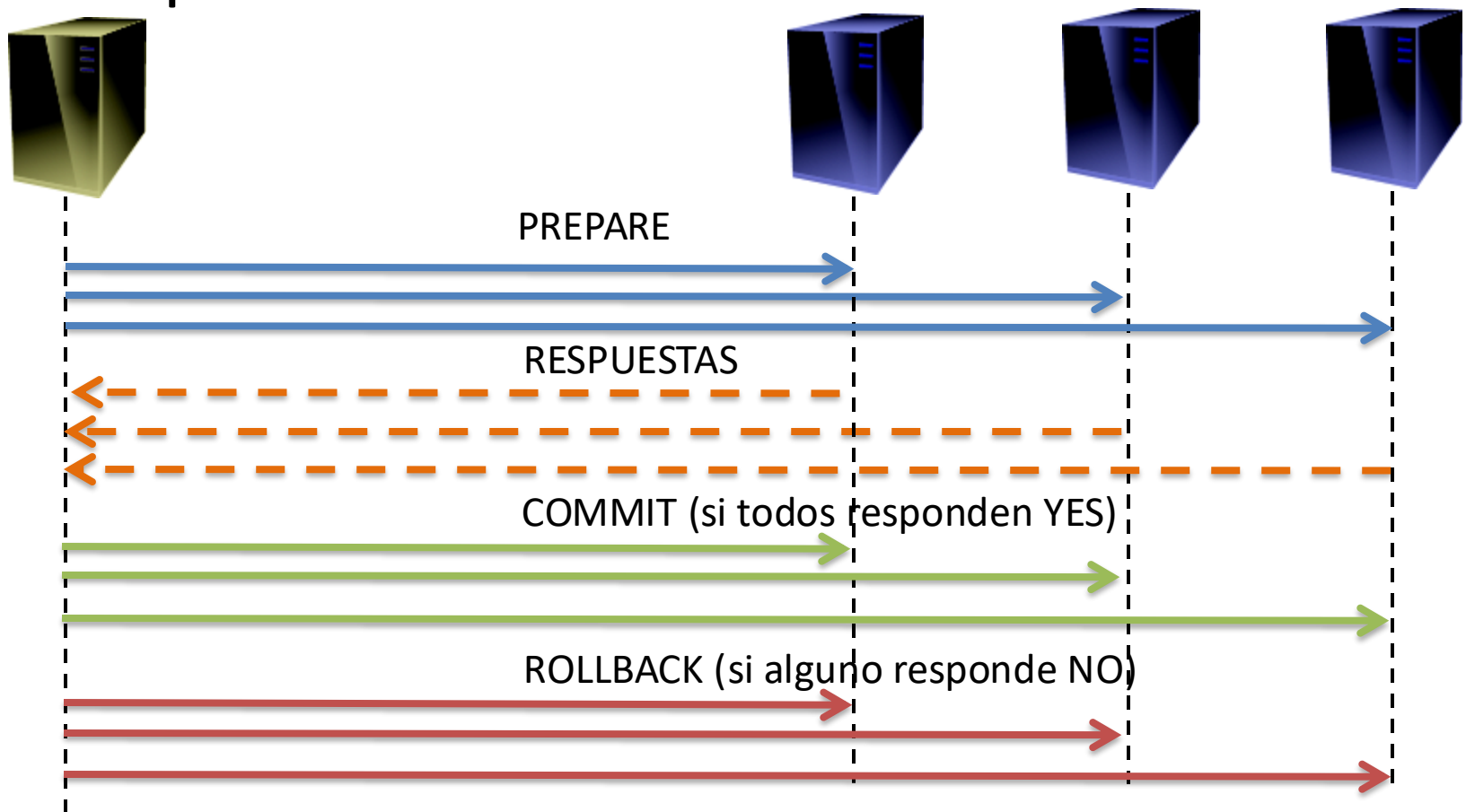
Dar de baja

- El servicio se da de baja temporalmente mientras se repara el Defecto
 - Desconectar, reparar y conectar
 - Reiniciar
 - Apagar y prender



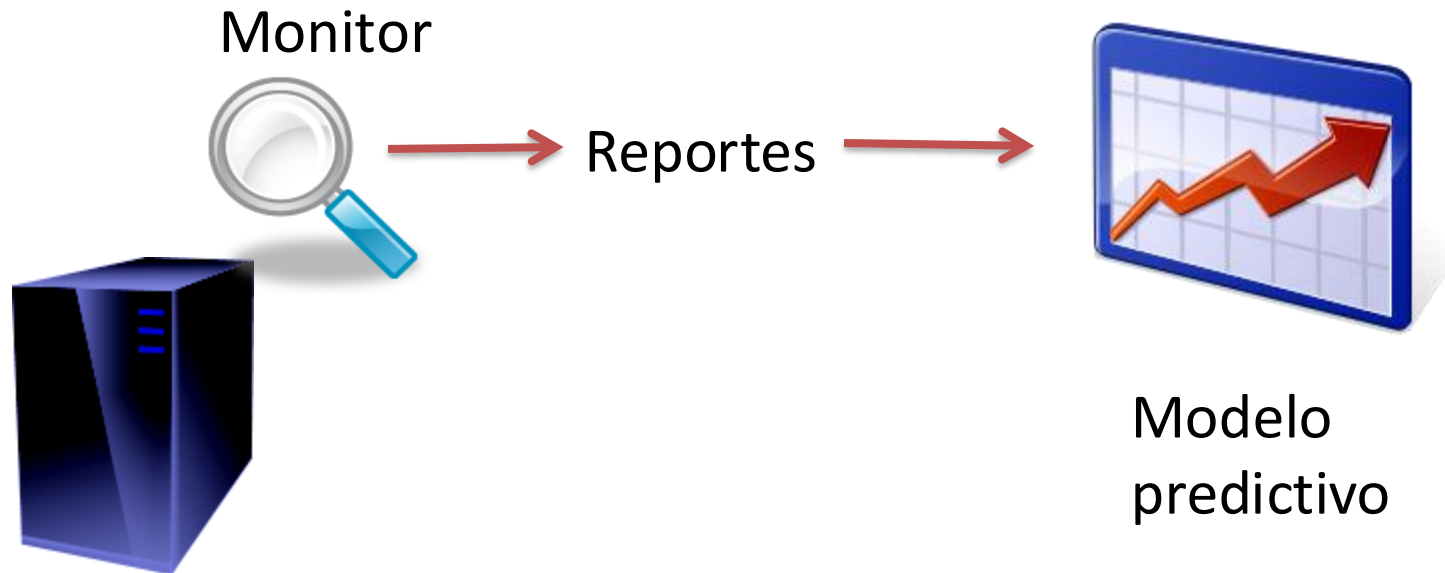
Transacciones

- ACID
- Two-phase commit <https://www.postgresql.org/docs/current/two-phase.html>



- Patrón SAGA

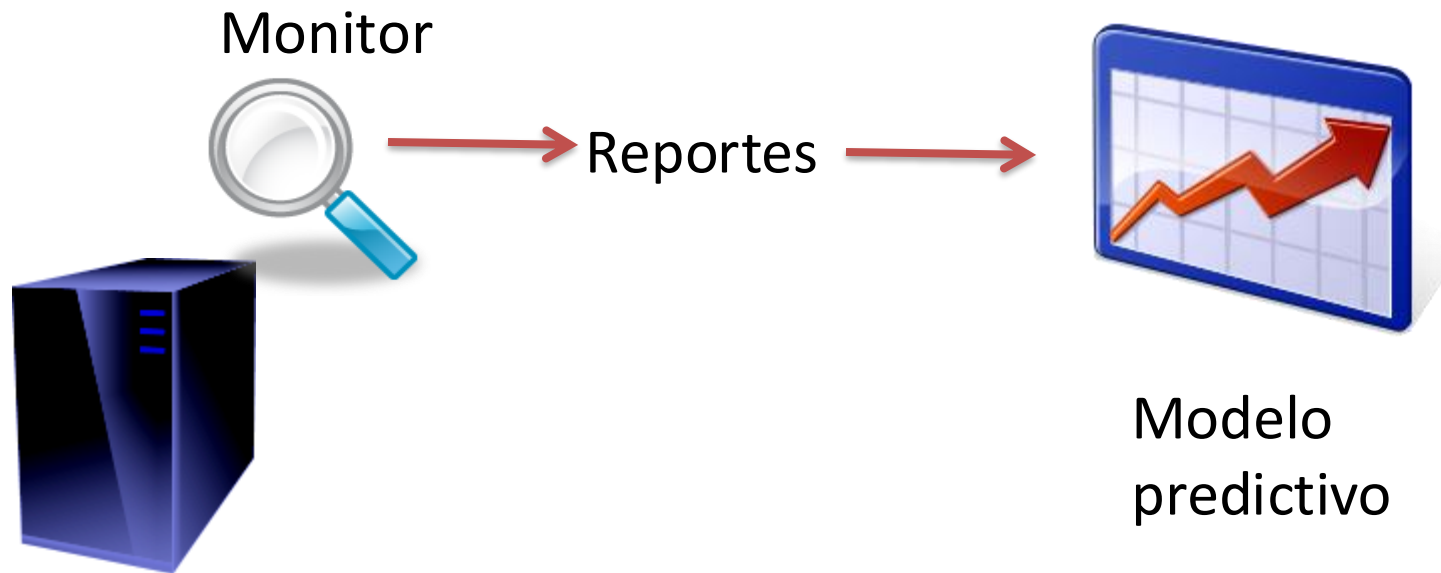
Modelo predictivo



- Ejemplos de técnicas:

- Cadenas de Markov
- Redes de Petri
- Árboles de Fallas
- Machine Learning

Modelo predictivo



Prevención de excepciones

- Clases de Excepciones (ej. Java)
- A más bajo nivel:
 - Smart pointers (C++)
 - Garbage collection (Java y otros)
 - Ownership model (Rust)

Incrementar estados "competentes"

- Un sistema es "incompetente" cuando no puede manejar un flujo alternativo
- Diseñar el sistema para que maneje más flujos alternativos -> aumentar "competencia"
 - Flujos normales
 - **Excepciones**

Patrones para Disponibilidad

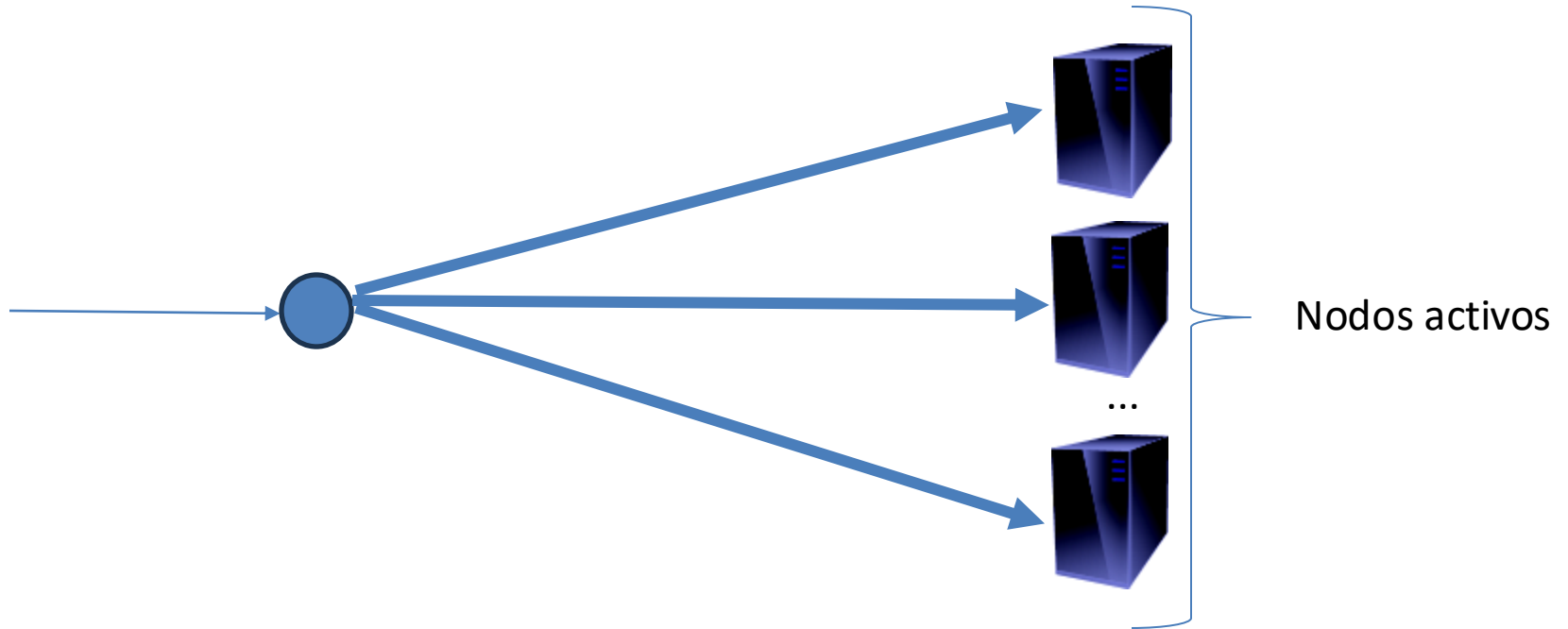
Patrones para Disponibilidad

- Redundancia
 - Redundancia activa (hot spare)
 - Redundancia pasiva (warm spare)
 - Repuesto (cold spare)
 - Triple Modular Redundancy (TMR)
- Circuit Breaker

Redundancia

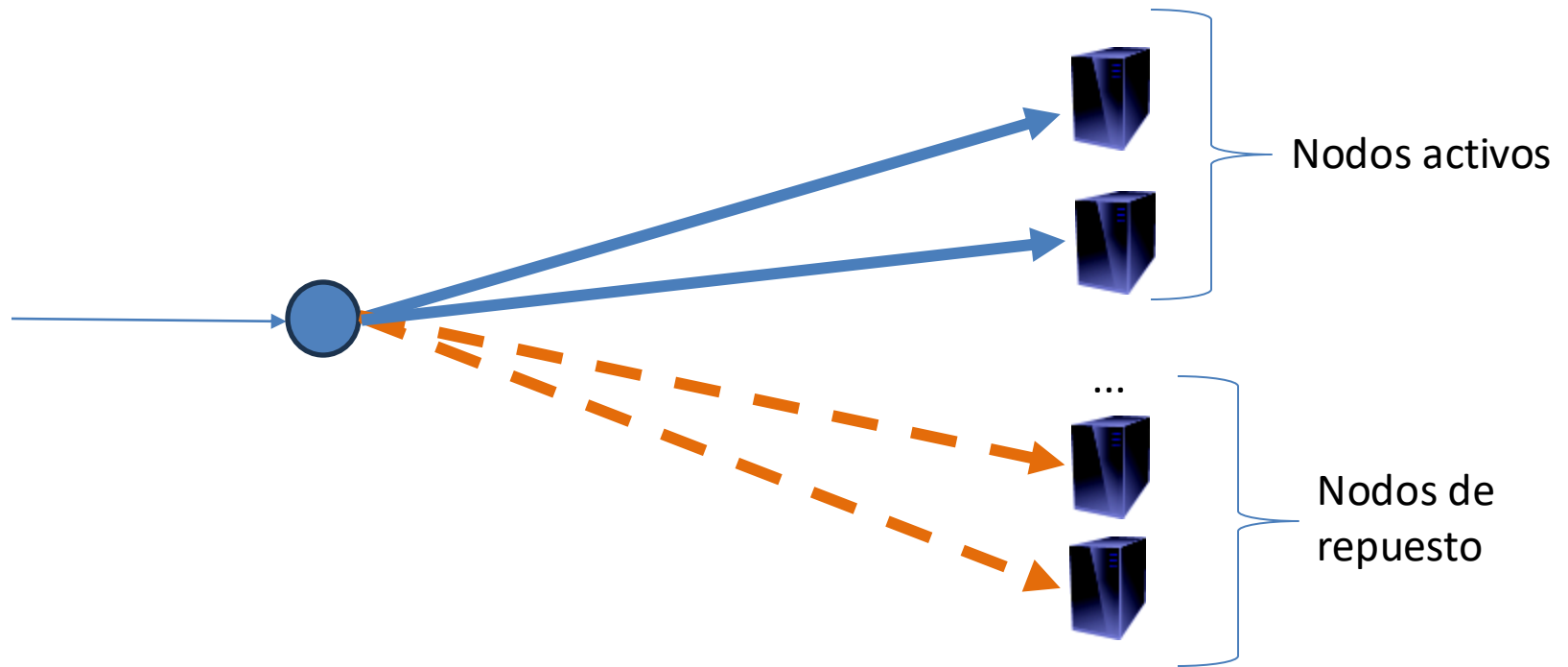
- Tener copias del nodo
- Activas + Repuestos
- ¿Cómo determinar si nodo activo está caído?
 - Monitoreo
 - Ping
 - Heartbeat
 - Sanity check
 - Voting
 - Etc.
- ¿Cómo sincronizar estados?
 - Checkpoints
 - Transacciones

Redundancia activa (hot spare)



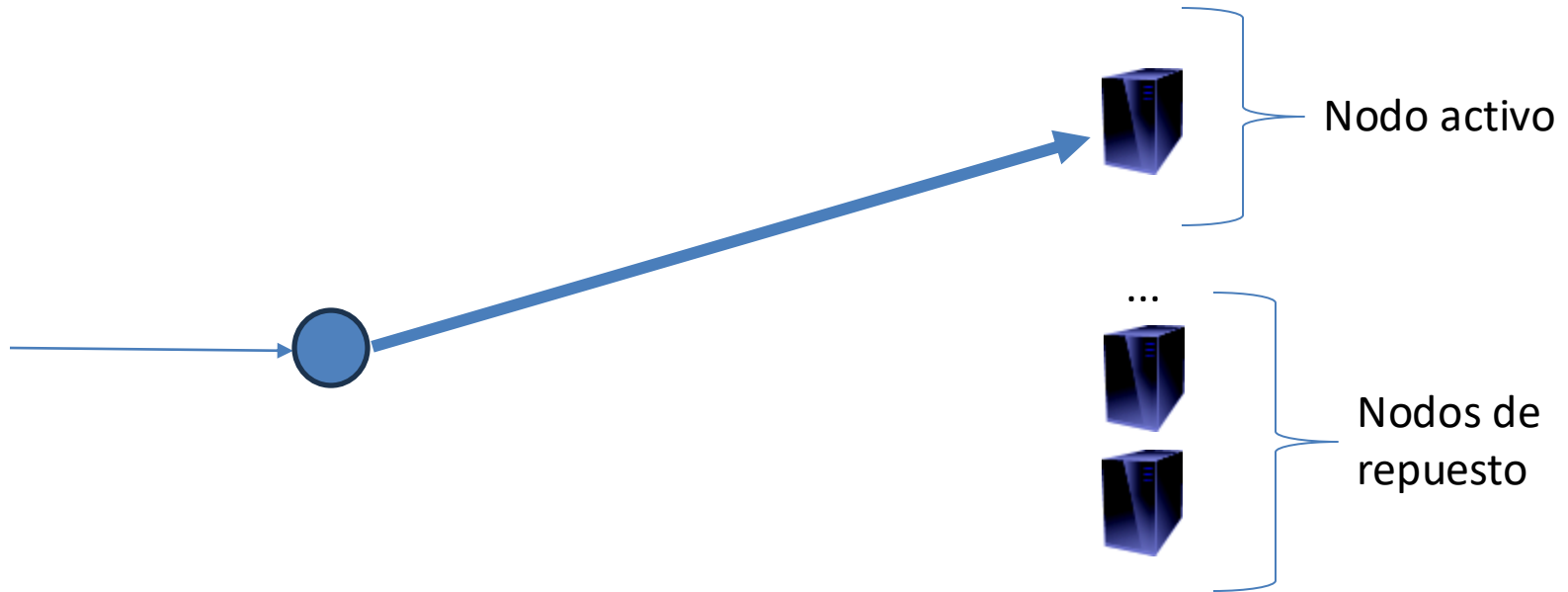
- Todos los nodos están activos simultáneamente (Atienden peticiones)
- Si uno falla, el otro puede fácilmente tomar su lugar
- Sincronización constante de estado
- Alto costo - Alta disponibilidad

Redundancia pasiva (warm spare)



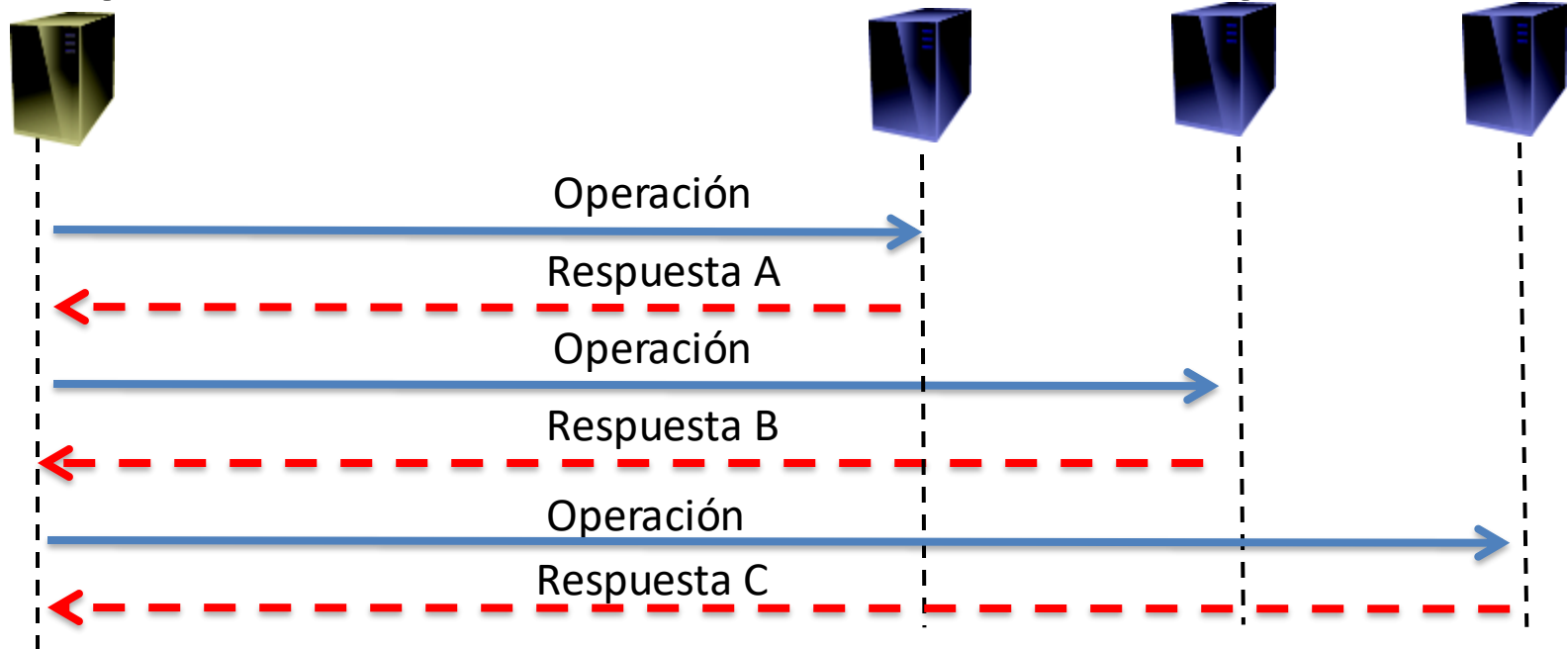
- Subconjunto "hot", activo
- Subconjunto "warm", modo shadow
 - Actualizaciones de estado más esporádicas
- Costo medio – media disponibilidad

Repuesto (cold spare)



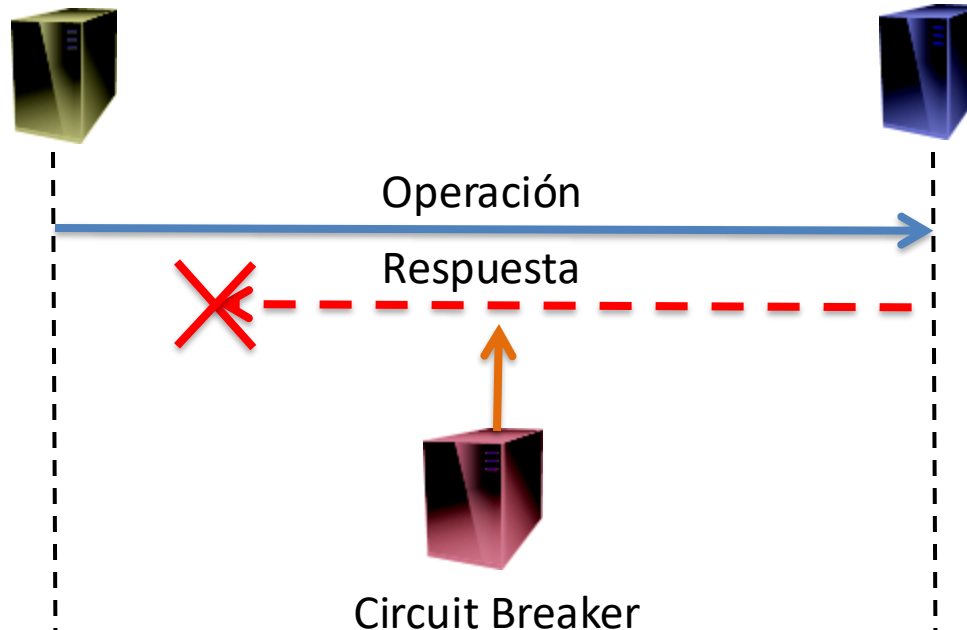
- Nodos de repuesto están apagados
- Se encienden cuando el nodo activo no funciona
- Menor costo – baja disponibilidad

Triple Modular Redundancy (TMR)



- 3+ sistemas redundantes
- Responden al tiempo a la misma operación crítica
- Respuesta correcta por mayoría
- Los dos correctos toman precedencia sobre el fallido
- Ejemplo: aviónica (caso Boeing), automatización industrial

Circuit Breaker



- Monitor externo al proceso
- Controla
 - Frecuencia de reintentos
 - Máximo preestablecido de reintentos
- Ejemplo: Resilience4j

Ejercicio

- Identifique tácticas y patrones relevantes para los escenarios de Disponibilidad identificados en su proyecto